WEEK 9 · PHASE 4

Lakeflow Jobs

Orchestration

DAG · Tasks · Retries · Triggers · Control Flow

- Task types: Notebook, SQL, Pipeline, Dashboard, Python
- DAG dependencies and run order
- Retry policies and timeout configuration
- Conditional branching and loops (if/else/for-each)
- Trigger types: scheduled, file arrival, table update
- Practice Questions (10 Qs)

Table of Contents

1. Lakeflow Jobs Overview 3
2. Task Types 4
3. DAG Dependencies 6
4. Retry & Timeout Policies 8
5. Conditional Branching 10
6. For-Each Task (Loops) 12
7. Trigger Types 13
8. Job Parameters & Widgets 15
9. Practice Questions 16
10. Answer Key 19

WEEK 9 **Lakeflow Jobs Overview** Databricks workflow orchestration engine

1. Lakeflow Jobs Overview

Lakeflow Jobs (formerly Databricks Workflows) is the native orchestration engine for running data pipelines on Databricks. A Job consists of one or more Tasks arranged in a DAG (Directed Acyclic Graph). Jobs can be triggered on a schedule, by external events, or run manually via API or UI.

Concept	Description
Job	Top-level orchestration unit — contains tasks, schedule, cluster config
Task	Individual unit of work (notebook, SQL query, DLT pipeline, etc.)
Run	A single execution instance of a Job
DAG	Directed Acyclic Graph — defines execution order and dependencies
Cluster	Job cluster (ephemeral) or existing all-purpose cluster
Job Cluster	Spun up for the job, terminated after — cost-efficient for production
All-Purpose Cluster	Shared cluster — higher cost, useful for development/debugging
Run-as Identity	Service principal or user identity under which the job runs

■ **Exam Focus** Exam: Production jobs should use Job Clusters (ephemeral, cheaper). All-purpose clusters are for interactive development only.

2. Task Types

Lakeflow Jobs supports multiple task types. Each type runs a different kind of workload. Knowing which task type to choose for a given scenario is a key exam topic.

Task Type	Runs	Use Case	Key Config
Notebook	A Databricks notebook (Python/SQL/Scala/R)	ETL logic, data exploration	Notebook path + parameters
SQL	A SQL query or SQL file	Reporting queries, table maintenance	SQL warehouse ID required
Delta Live Tables Pipeline	A DLT pipeline definition	Streaming/batch medallion pipelines	Pipeline ID
Python Script	A .py file from workspace/repo	Standalone Python jobs, ML scripts	Script path + parameters
dbt	A dbt project in a Git repo	SQL transformation with dbt	dbt project path, profiles
Dashboard	Refreshes a Lakeflow dashboard	Scheduled report generation	Dashboard ID
Spark Submit	spark-submit compatible job	Legacy/Spark apps, JARs	Main class or script
Run Job	Triggers another Lakeflow Job	Composing jobs, reusable pipelines	Target Job ID

2.1 Notebook Task — Most Common

■ **Notebook task JSON config**

```
// Notebook task configuration (JSON API)
{
  "task_key": "ingest_bronze",
  "notebook_task": {
    "notebook_path": "/Repos/team/project/notebooks/01_ingest",
    "base_parameters": {
      "env": "prod",
      "date": "{{job.start_time.iso_date}}"
    }
  },
  "job_cluster_key": "bronze_cluster",
  "timeout_seconds": 3600,
  "max_retries": 2
}
```

2.2 SQL Task — Requires SQL Warehouse

■ **SQL task JSON config**

```
// SQL task — requires a SQL warehouse (not a cluster)
{
  "task_key": "refresh_gold_view",
  "sql_task": {
    "query": {
      "query_id": "abc123def456"
    },
    "warehouse_id": "warehouse_xyz"
  }
}
```

```
// Alternatively — run a SQL file from a Git repo
"sql_task": {
  "file": {"path": "/sql/refresh_gold.sql"},
  "warehouse_id": "warehouse_xyz"
}
```

■ **Exam Focus** SQL Tasks REQUIRE a SQL Warehouse — they cannot run on a cluster. Notebook Tasks with SQL notebooks CAN run on a cluster.

3. DAG Dependencies

Tasks are arranged in a DAG using depends_on. A task runs only after all its upstream dependencies have succeeded. Databricks automatically parallelizes tasks with no dependencies between them.

3.1 depends_on — Defining Task Order

■ **Linear DAG — depends_on**

```
// Three-task DAG: ingest → transform → publish
//
// [ingest_bronze]
//       |
// [transform_silver]
//       |
// [publish_gold]

"tasks": [
  {"task_key": "ingest_bronze", ...},

  {"task_key": "transform_silver",
   "depends_on": [{"task_key": "ingest_bronze"}],
   ...},

  {"task_key": "publish_gold",
   "depends_on": [{"task_key": "transform_silver"}],
   ...}
]
```

3.2 Parallel Tasks

■ **Fan-out fan-in DAG**

```
// Fan-out: ingest → [transform_A, transform_B] → publish
//
//         [ingest]
//        /        \
// [transform_A]  [transform_B]  ← run in PARALLEL
//        \        /
//         [publish]

"tasks": [
  {"task_key": "ingest", ...},

  {"task_key": "transform_A",
   "depends_on": [{"task_key": "ingest"}], ...},

  {"task_key": "transform_B",
   "depends_on": [{"task_key": "ingest"}], ...},

  {"task_key": "publish",
   "depends_on": [
     {"task_key": "transform_A"},
     {"task_key": "transform_B"}   // waits for BOTH
   ], ...}
]
```

3.3 depends_on with Outcome Conditions

By default, a downstream task runs only if ALL upstream tasks succeeded. You can override this with outcome-based dependencies:

■ depends_on with outcome condition

```
"depends_on": [
  {
    "task_key": "risky_task",
    "outcome": "FAILED"   // run THIS task only if risky_task FAILED
  }
]

// outcome values:
// "SUCCEEDED"  — run if upstream succeeded (default)
// "FAILED"     — run if upstream failed (e.g., error handler task)
// "TIMEDOUT"   — run if upstream timed out
// "SKIPPED"    — run if upstream was skipped
```

■ **Exam Focus** outcome="FAILED" is the key pattern for error-handling tasks — run a cleanup or alerting task only when a specific upstream task fails.

4. Retry & Timeout Policies

Retries and timeouts make pipelines fault-tolerant. Each task can have its own independent retry and timeout settings.

Setting	Scope	Description	Default
max_retries	Per task	Number of automatic retries on failure	0 (no retries)
min_retry_interval_millis	Per task	Minimum wait between retries (ms)	0 ms
retry_on_timeout	Per task	Whether to retry if task times out	false
timeout_seconds	Per task	Max seconds before task is killed	0 = no timeout
run_if	Per task	Condition to run task (ALL_SUCCESS, etc.)	ALL_SUCCESS

■ **Retry and timeout task config**

```
// Retry configuration on a task
{
  "task_key": "transform_silver",
  "max_retries": 3,
  "min_retry_interval_millis": 60000,   // 1 minute between retries
  "retry_on_timeout": true,
  "timeout_seconds": 1800,               // 30 minutes max
  "depends_on": [{"task_key": "ingest"}],
  "notebook_task": {...}
}
```

4.1 run_if — Conditional Execution

run_if Value	Task Runs When
ALL_SUCCESS (default)	All upstream tasks succeeded
AT_LEAST_ONE_SUCCESS	At least one upstream task succeeded
NONE_FAILED	No upstream tasks failed (succeeded or skipped)
ALL_DONE	All upstream tasks finished (any outcome)
AT_LEAST_ONE_FAILED	At least one upstream task failed
ALL_FAILED	All upstream tasks failed

■ **Tip** run_if="ALL_DONE" is used for cleanup/notification tasks that should always run regardless of whether upstream succeeded or failed.

5. Conditional Branching (If/Else)

The If/Else task type evaluates a condition at runtime and routes execution to one of two branches. This enables dynamic pipelines that behave differently based on data or environment conditions.

5.1 If/Else Task — Basic Pattern

■ **If/Else task config**

```
// If/Else task evaluates a condition expression
{
  "task_key": "route_by_env",
  "condition_task": {
    "op": "EQUAL_TO",
    "left": "{{job.parameters.env}}",
    "right": "prod"
  }
}

// Branch tasks use outcome-based depends_on:
{"task_key": "prod_path",
 "depends_on": [{"task_key": "route_by_env", "outcome": "true"}]}

{"task_key": "dev_path",
 "depends_on": [{"task_key": "route_by_env", "outcome": "false"}]}
```

5.2 Condition Operators

Operator	Description	Example
EQUAL_TO	Left == Right	env == "prod"
NOT_EQUAL_TO	Left != Right	env != "dev"
GREATER_THAN	Left > Right (numeric)	record_count > 0
LESS_THAN	Left < Right (numeric)	error_count < 10
GREATER_THAN_OR_EQUAL_TO	Left >= Right	rows >= 1000
LESS_THAN_OR_EQUAL_TO	Left <= Right	latency_ms <= 5000

5.3 Practical: Skip Gold if No Silver Rows

■ **Conditional skip pattern**

```
// Pattern: check row count, skip gold if empty

// Task 1: Count silver rows, store in task value
Task: count_silver (Notebook)
  dbutils.jobs.taskValues.set("row_count",
    spark.table("silver.orders").count())

// Task 2: If/Else — check count
"condition_task": {
  "op": "GREATER_THAN",
  "left": "{{tasks.count_silver.values.row_count}}",
  "right": "0"
}

// Task 3a: Build gold (outcome: true)
// Task 3b: Send empty-run notification (outcome: false)
```

6. For-Each Task (Loops)

The For-Each task iterates over a list of values and runs a child task for each element — enabling parallel or sequential processing of dynamic sets (e.g., process one file per iteration, one country per iteration).

■ **For-Each task config**

```
// For-Each task config
{
  "task_key": "process_all_regions",
  "for_each_task": {
    "inputs": "{{job.parameters.regions}}",
    // inputs: JSON array string: ["US","EU","APAC"]
    "concurrency": 3,
    // concurrency: how many child tasks run in parallel (default 1)
    "task": {
      "task_key": "process_region",
      "notebook_task": {
        "notebook_path": "/notebooks/process_region",
        "base_parameters": {
          "region": "{{input}}"   // {{input}} = current iteration value
        }
      }
    }
  }
}
```

For-Each Setting	Description
inputs	JSON array string — each element becomes one iteration
concurrency	How many child tasks run simultaneously (1 = sequential)
{{input}}	Built-in reference to current iteration value in child task
task	The child task definition (inline — any task type)

■ **Tip** For-Each concurrency > 1 runs iterations in parallel. Set concurrency = 1 for sequential processing (e.g., ordered file processing).

7. Trigger Types

Lakeflow Jobs supports multiple trigger types for starting job runs automatically. Choosing the right trigger depends on the data arrival pattern and latency requirements.

Trigger Type	Config	When Job Runs	Use Case
Scheduled (cron)	Cron expression	At fixed time intervals	Daily/hourly batch jobs
File Arrival	Cloud storage path + pattern	When new files land in path	Event-driven ingestion from S3/ADLS
Table Update	Delta table name	When a Delta table is updated	Downstream processing after upstream write
Continuous	No schedule	Runs immediately after previous run completes	Near-real-time processing loops
Manual	API / UI button	On-demand	Testing, ad-hoc re-runs

7.1 Scheduled Trigger — Cron Expression

■ **Scheduled cron trigger**

```
// Quartz cron syntax: sec min hr day-of-month month day-of-week year

"schedule": {
  "quartz_cron_expression": "0 0 6 * * ?",  // Every day at 06:00 UTC
  "timezone_id": "America/New_York"
}

// Common cron patterns:
// "0 0 * * * ?"    — Every hour
// "0 0 6 * * ?"    — Daily at 6:00 AM
// "0 0 6 ? * MON"  — Every Monday at 6:00 AM
// "0 0 6 1 * ?"    — 1st of every month at 6:00 AM
// "0 */15 * * * ?" — Every 15 minutes
```

7.2 File Arrival Trigger

■ **File arrival trigger**

```
"trigger": {
  "file_arrival": {
    "url": "s3://my-bucket/incoming/orders/",
    "min_time_between_triggers_seconds": 300,  // 5 min debounce
    "wait_after_last_change_seconds": 60       // stability wait
  }
}

// Job runs when new file(s) appear in the path
// Built-in file deduplication — same file will not trigger twice
```

7.3 Table Update Trigger

■ **Table update trigger**

```
"trigger": {
  "table": {
    "condition": "ANY_UPDATED",
    "table_names": [
      "catalog.silver.orders",
      "catalog.silver.customers"
    ],
    "min_time_between_triggers_seconds": 600
```

```
  }
}

// Trigger options: ANY_UPDATED | ALL_UPDATED
// Monitors Delta table transaction log for new commits
```

■ **Note** Table Update triggers use the Delta transaction log to detect changes — no polling needed. They fire when ANY commit appears on the monitored table(s).

8. Job Parameters & Task Values

Jobs can pass parameters to tasks and tasks can share computed values with downstream tasks using the Task Values API. This enables dynamic, data-driven pipelines.

8.1 Job Parameters — Pass Values to Tasks

■ Job parameters passing

```
// Define job parameters in job config
"parameters": [
  {"name": "env",       "default": "dev"},
  {"name": "load_date", "default": "{{job.start_time.iso_date}}"}
]

// Reference in notebook task base_parameters
"base_parameters": {
  "env":       "{{job.parameters.env}}",
  "load_date": "{{job.parameters.load_date}}"
}

// Access in notebook with dbutils.widgets
env = dbutils.widgets.get("env")
load_date = dbutils.widgets.get("load_date")
```

8.2 Task Values — Share Between Tasks

■ Task Values API

```
# Task A: Compute and store a value
row_count = spark.table("silver.orders").count()
dbutils.jobs.taskValues.set(key="row_count", value=row_count)

# Task B: Read value from Task A
count = dbutils.jobs.taskValues.get(
    taskKey="task_a",
    key="row_count",
    default=0,
    debugValue=999  # value used in interactive notebook runs
)

# Reference in If/Else condition:
# "{{tasks.task_a.values.row_count}}"
```

QUIZ **Practice Questions** Week 9 — Lakeflow Jobs Orchestration

9. Practice Questions

1. A Lakeflow Job has tasks A → B → C. Task B fails. What happens to Task C by default?

A) Task C runs with B's partial results

B) Task C is skipped (not run)

C) Task C runs regardless of B's outcome

D) The entire job is retried from Task A

2. A task must run a SQL query against a Delta table. Which Lakeflow task type is required, and what compute resource does it need?

A) Notebook task — uses a cluster

B) SQL task — requires a SQL Warehouse

C) SQL task — uses a job cluster

D) Python Script task — any cluster

3. Two tasks (Transform_A and Transform_B) both depend on Ingest. Transform_A and Transform_B have no dependency between them. How does Databricks execute them?

A) Transform_A runs first, then Transform_B sequentially

B) Transform_A and Transform_B run in parallel after Ingest succeeds

C) The order is random — Databricks picks one

D) Both run simultaneously with Ingest

4. A cleanup task should always run after the main ETL task, regardless of whether ETL succeeded or failed. Which run_if setting achieves this?

A) run_if="ALL_SUCCESS"

B) run_if="AT_LEAST_ONE_FAILED"

C) run_if="ALL_DONE"

D) run_if="NONE_FAILED"

5. A job parameter references "{{job.start_time.iso_date}}". What does this resolve to at runtime?

A) The timestamp when the job was created

B) The ISO date string of the current job run start time

C) A static default value set in the job definition

D) The date the notebook was last modified

6. A For-Each task processes 10 regions with concurrency=5. How many child tasks run simultaneously?

A) 1 — For-Each is always sequential

B) 5 — limited by concurrency setting

C) 10 — all iterations run at once

D) Determined by cluster core count

7. Task A stores a value with dbutils.jobs.taskValues.set("count", 500). Task B reads it with dbutils.jobs.taskValues.get(taskKey="task_a", key="count"). What is the correct depends_on for Task B?

A) No depends_on needed — task values are globally accessible

B) Task B must depend on Task A to guarantee the value is set before Task B reads it

C) Task B must depend on the job trigger, not Task A

D) Task values are read-only in the job definition, not at runtime

8. A file arrival trigger is set on s3://bucket/incoming/. A new file arrives every 5 minutes. min_time_between_triggers_seconds=300. How often does the job run?

A) Every 5 minutes — once per file

B) At most every 5 minutes — multiple files may be batched into one run

C) Continuously — once the previous run finishes

D) Never — file arrival triggers require SNS setup

9. An If/Else task uses op="GREATER_THAN" with left="{{tasks.count_task.values.rows}}" and right="0". Task A depends on outcome="true". Task B depends on outcome="false". When rows=0, which task runs?

A) Task A (true branch)

B) Task B (false branch)

C) Both Task A and Task B

D) Neither — the job fails

10. Which cluster type should a production Lakeflow Job use to minimize cost?

A) All-purpose cluster — always available, no startup delay

B) Job cluster — ephemeral, starts for the job and terminates after

C) SQL Warehouse — cheapest option for any task type

D) Shared cluster — shared across all users reduces cost

ANSWERS **Answer Key** Week 9 Practice Questions

10. Answer Key

#	Ans	Explanation
1	(B)	By default, run_if="ALL_SUCCESS". If upstream Task B fails, Task C is skipped (not executed). The job run is marked FAILED.
2	(B)	SQL tasks require a SQL Warehouse (serverless or pro). They cannot run on a cluster. Notebook tasks with SQL code can run on a cluster, but the SQL task type specifically needs a warehouse.
3	(B)	When two tasks share the same upstream dependency but have no dependency between themselves, Databricks runs them in parallel — maximizing throughput.
4	(C)	run_if="ALL_DONE" runs the task after all upstream tasks have finished, regardless of their outcome (success, failure, timeout, or skipped). Perfect for cleanup/notification tasks.
5	(B)	{{job.start_time.iso_date}} is a dynamic reference that resolves to the ISO date (YYYY-MM-DD) of the current job run's start time — useful for date-partitioned loads.
6	(B)	concurrency=5 means up to 5 child task iterations run simultaneously. The remaining 5 wait until a slot is free. Default concurrency=1 (sequential).
7	(B)	Task B must have depends_on Task A. Without the dependency, Task A and B might run in parallel, and the value may not be set when Task B tries to read it.
8	(B)	min_time_between_triggers_seconds=300 (5 min) means the job runs at most once per 5 minutes. Multiple files arriving in that window are processed in the same run.
9	(B)	GREATER_THAN: 0 > 0 is false. outcome="false" branch (Task B) runs. Task A (outcome="true") is skipped. The job continues normally.
10	(B)	Job clusters are ephemeral — created for the job run and terminated afterward. This eliminates idle cost. All-purpose clusters run continuously and are expensive for production use.

11. Week 9 Cheat Sheet

Concept	Key Facts
Task Types	Notebook / SQL / Pipeline / Python / dbt / Dashboard / Spark Submit / Run Job
SQL Task	Requires SQL Warehouse — NOT a cluster
Job Cluster	Ephemeral — create at start, terminate after — use in production
All-Purpose Cluster	Persistent — expensive — use in development only
depends_on	Defines DAG order; default: run after upstream SUCCEEDED
outcome=	"SUCCEEDED"|"FAILED"|"TIMEDOUT"|"SKIPPED" on depends_on
run_if	ALL_SUCCESS(default)|AT_LEAST_ONE_SUCCESS|NONE_FAILED|ALL_DONE|AT_LEAST_ONE_FAILED
max_retries	Number of auto-retries on task failure (default 0)
retry_on_timeout	Also retry if task exceeds timeout_seconds
If/Else task	Condition: op + left + right; branches via outcome="true"|"false"
For-Each task	inputs=JSON array; concurrency=N parallel; {{input}}=current value
Scheduled trigger	Quartz cron expression + timezone_id
File Arrival trigger	Monitors cloud path; min_time_between_triggers_seconds debounce
Table Update trigger	Monitors Delta table commits; ANY_UPDATED or ALL_UPDATED
Job parameters	{{job.parameters.name}}; {{job.start_time.iso_date}}
Task Values	dbutils.jobs.taskValues.set/get; {{tasks.X.values.key}} in conditions

BONUS **Advanced Job Patterns & Exam Scenarios** Deep Dive

A. Job Repair & Re-run

When a job run fails, Databricks allows repairing the run — re-running only the failed tasks without re-running already-successful tasks. This saves time and compute cost.

■ **Repair run via API**

```
// Repair a failed run via API
POST /api/2.1/jobs/runs/repair
{
  "run_id": 123456,
  "rerun_tasks": ["failed_task_key"],
  "latest_repair_id": null  // null for first repair attempt
}

// Or via UI: Jobs → Run history → Failed run → Repair Run button
// Only failed + skipped tasks re-run; successful tasks keep their results
```

■ **Exam Focus** Repair Run is critical for large pipelines — avoids re-running expensive successful tasks. Re-run tasks start with their original parameters.

B. Job Cluster vs All-Purpose: Decision Matrix

Factor	Job Cluster	All-Purpose Cluster
Cost	Lower — pay only during run	Higher — pay while cluster is alive
Startup time	~3–5 minutes (cold start)	0 (already running)
Use case	Production scheduled jobs	Interactive dev, notebooks
Isolation	Dedicated to one job run	Shared across users
Libraries	Installed fresh per run	May have dependency conflicts
Databricks recommendation	Production jobs	Development only

C. Complete Task Control Flow Reference

Control Flow	Mechanism	Config Key
Sequential	A depends on B (linear chain)	depends_on
Parallel	Multiple tasks, same depends_on	depends_on (same upstream)
Conditional (If/Else)	Condition evaluates to true/false	condition_task + outcome
Loop (For-Each)	Iterate over array of values	for_each_task + inputs + concurrency
Error handling	Task runs when upstream fails	depends_on outcome="FAILED"
Always-run cleanup	Task runs regardless of outcome	run_if="ALL_DONE"
Skip on no data	Condition checks row count	If/Else + task value check
Pass data between tasks	Task Values API	dbutils.jobs.taskValues.set/get

D. Self-Assessment Checklist

- ■ I know all 8 Lakeflow task types and when to use each
- ■ I know SQL tasks require SQL Warehouse (not a cluster)
- ■ I can draw a fan-out fan-in DAG with depends_on
- ■ I know outcome values for depends_on: SUCCEEDED/FAILED/TIMEDOUT/SKIPPED
- ■ I know all 6 run_if values and their trigger conditions
- ■ I can configure max_retries, min_retry_interval_millis, retry_on_timeout
- ■ I understand If/Else task: op + left + right → outcome true/false
- ■ I know For-Each inputs/concurrency/{{input}} pattern
- ■ I know the 3 trigger types: scheduled (cron), file arrival, table update
- ■ I can pass job parameters and use dbutils.jobs.taskValues

■ **Exam Focus** Domain 4 (Lakeflow Jobs) = 16% of exam (~7 questions). Focus on DAG dependencies, run_if conditions, task types, and trigger configurations.

F. Lakeflow Jobs — Complete Exam Reference

Scenario	Correct Config
"Task B should only run if Task A fails"	depends_on Task A with outcome="FAILED"
"Cleanup task always runs"	run_if="ALL_DONE" on cleanup task
"Process 10 regions in parallel"	For-Each task, inputs=regions array, concurrency=10
"Job runs when new S3 file arrives"	file_arrival trigger with path and debounce
"Job runs when Delta table updated"	table trigger, condition="ANY_UPDATED"
"Pass today's date to notebook"	{{job.start_time.iso_date}} in base_parameters
"Share row count between tasks"	dbutils.jobs.taskValues.set("count", n)
"Retry 3 times with 1-min gap"	max_retries=3, min_retry_interval_millis=60000
"Cost-efficient production cluster"	job_cluster (ephemeral) not all_purpose
"SQL task needs ___"	A SQL Warehouse — not a cluster
"Re-run only failed tasks"	Repair Run feature in job run history

■ **Exam Focus** Domain 4 (Lakeflow Jobs) = 16% of exam. Trigger types, run_if values, task types (especially SQL task needing warehouse), and DAG dependency outcomes are the highest-yield topics.